IIT HYDERABAD

CS5903 – Distributed AI Training

AdaptEngine

3. Adaptive Learning, Diagnostic Assessment and Personalized Path-Planning Engine

Mid Report

Vuppula Hemanth Reddy

AI23BTECH11033

Group 15

Repository: https://github.com/vhemanth06/AdaptEngine

Contents

1 Problem Statement **3**

1.1 Motivation . . . 3

1.2 Project Milestones . . . 3

2 Proposed Approach **4**

2.1 Proposed Solution . . . 4

2.2 M1: Curriculum Representation . . . 4

2.3 M2: Adaptive Diagnostic . . . 5

2.4 M3: Time-Budgeted Planner . . . 6

2.5 M4: Mastery Model, Evidence State and Difficulty Policy . . . 6

2.5.1 Mastery model . . . 6

2.5.2 Evidence state . . . 7

2.5.3 Difficulty policy . . . 7

2.6 M5: Spaced Review and Remediation . . . 7

2.7 M6: Dynamic Replanning . . . 8

2.8 M7: Decision Traces and Explanations . . . 8

2.9 M8: Comparative Evaluation . . . 8

2.10 Optional Agentic Extension . . . 8

3 System Design **9**

3.1 Design Principles . . . 9

3.2 System Architecture . . . 9

3.3 System Context . . . 11

3.4 Major Components and Status . . . 11

3.5 Component Interaction and Workflow . . . 12

3.6 Inputs and Outputs . . . 13

3.7 Data Model . . . 14

4 Implementation **14**

4.1 Work Completed Until Week 2: Foundation . . . 14

4.2 Work Completed in Week 3 . . . 15

4.2.1 Fixes following the Week 2 mentor review . . . 15

4.2.2 Adaptive diagnostic (M2) . . . 15

5 Current Progress **15**

5.1 Milestone Progress . . . 15

5.2 Acceptance Test Register 16

6 Assumptions and Limitations **18**

7 Future Work **18**

1 Problem Statement

Online courses usually present lessons in one fixed order for every learner. In reality, learners differ a lot. Some have never seen the topic, some already know half of it, and their deadlines and free time also vary. A fixed sequence cannot suit both a beginner who has only 10 minutes a day and an experienced person who wants a short refresher. The first learner gets overwhelmed by lessons that are too hard or too long, while the second wastes time on things they already know.

1.1 Motivation

The motivation behind AdaptEngine is to replace this one-size-fits-all sequence with a personalization engine. Such an engine should find out what a learner already knows, plan what to study within their goal, deadline and daily time limit, and keep adjusting as they progress. It must do this while always respecting the order in which topics depend on each other. It should also explain why each activity was chosen, so that the learner can trust the plan.

Building this is difficult for a few reasons:

- The initial skill level has to be estimated from a very short diagnostic, and the estimate must carry uncertainty instead of being a single number.
- Prerequisites are strict rules, whereas time, deadline and difficulty are things to optimize. If a goal cannot fit in the available time, the system must say so clearly and not silently break a rule.
- New lessons, remediation and spaced review all compete for the same few minutes each day, so a clear priority policy is needed.
- After missed work or changed constraints, the new plan must remain valid and keep everything already completed.
- Every inclusion, omission, review or difficulty change must be traceable to learner state, graph constraints or a policy score.

1.2 Project Milestones

AdaptEngine is a backend policy and recommendation engine for any subject whose content can be organised as a prerequisite graph of concepts and whose exercises can be graded as correct or incorrect. The prototype is demonstrated on a small Python curriculum, but the engine itself does not depend on the subject. It is used through a Python API and a thin command-line interface. For a given learner, the complete system is designed to:

- **Represent the curriculum** as a prerequisite graph of concepts, lessons and exercises with durations (M1).
- **Estimate mastery** of each concept, together with its uncertainty, using a short bounded adaptive diagnostic (M2).
- **Generate a day-by-day study plan** that never breaks prerequisites and fits the learner's goal, deadline and daily budget of 5, 10, 15 or 30 minutes, or reports clearly that the goal is infeasible (M3).
- **Update mastery** after every quiz or exercise and **adapt difficulty** using an explicit rule (M4).

- **Schedule spaced review and remediation** based on forgetting risk, prerequisite importance and remaining daily time (M5).
- **Replan** when the learner misses a day, fails an assessment, or changes the deadline or daily budget, while keeping completed work (M6).
- **Explain every decision** through machine-readable decision traces (M7).
- **Evaluate it** against fixed, static and greedy baselines on diagnostic accuracy, plan feasibility, runtime, prediction and calibration, and simulated learning gains (M8).

The full learning platform, real course authoring and private learner data are outside the scope of this project. The aim is a working, documented prototype and an evaluation report that shows different valid plans for synthetic learners with different skills, deadlines and time budgets, together with a measurable improvement over non-adaptive baselines.

2 Proposed Approach

2.1 Proposed Solution

AdaptEngine is designed as a deterministic engine in which a mastery model estimates what the learner knows and a set of hard constraints decides what is allowed. The engine answers three questions for every learner: *what does this learner already know*, *what should they do next within their time*, and *why*. The overall pipeline is

$$\begin{aligned}&\text{validated curriculum} \to \text{diagnose} \to \text{eligible activities} \to \text{deterministic ranking}\\ &\to \text{budgeted schedule} \to \text{events} \to \text{replan}.\end{aligned}$$

The central design rule is the separation between **validity** and **preference**:

- **Hard constraints** (prerequisites and the daily time budget) are checked by the engine and can never be traded away.
- **Soft objectives** (review urgency, path unlocking, difficulty fit) are scores used only to *rank* activities that have already passed the hard filter.

A ranking score is therefore never responsible for checking whether a rule is violated. If a goal cannot be met, the engine reports it as infeasible and does not silently break a rule.

The approach is described below milestone by milestone (M1–M8), followed by the optional agentic extension. The mastery model and evidence state are described under M4, since the diagnostic (M2), planner (M3) and review (M5) all build on them.

2.2 M1: Curriculum Representation

The curriculum is a directed acyclic graph (DAG) in which nodes are concepts and edges are mandatory prerequisites. Each concept has lessons and exercises, and each activity has a target concept, an indivisible integer duration in minutes, and (for exercises) a difficulty $d \in [0.05, 0.95]$. The graph is handled by NetworkX for topological order, ancestor queries and cycle detection.

Content is trusted only after passing a gate: schema validation, rejection of unknown prerequisite references and cycles, and a review check against a SHA-256 manifest. This gate is applied at every live entry point so that altered or unreviewed content cannot reach the decision logic.

The pipeline is

$$\text{draft content} \rightarrow \text{human review} \rightarrow \text{hash recorded} \rightarrow \text{live curriculum}.$$

2.3 M2: Adaptive Diagnostic

The diagnostic must give a calibrated starting estimate from very few questions. It uses the mastery model and posteriors defined under M4 (Section 2.5).

Uncertainty as entropy. Uncertainty is measured using the entropy of the mastery probability,

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p),$$

where p is the probability that a student has mastered a skill. Entropy is highest at $p = 0.5$ and lowest at $p = 0$ or $p = 1$. It is used to guide item selection and stopping, subject to a fixed maximum number of items. Since entropy measures uncertainty rather than estimation accuracy, mastery estimates are evaluated separately using Mean Absolute Error (MAE) and the Brier score on simulated student attempts not used for model development.

Diagnostic rules. The proposed rules are:

- **Cold start:** every concept begins at $p_0 = 0.20$ and is marked `UNASSESSED`, which is treated as unmastered for diagnostic purposes. Existing learner state is preserved and never overwritten.
- **Concept order:** concepts are visited in stable topological order. If a concept ends with $p < 0.40$, its descendants are not probed and remain `UNASSESSED`, since their prerequisite is not met.
- **Item selection (expected information gain):** For the active concept, the diagnostic selects an unasked exercise with the highest expected reduction in entropy:

$$\text{EIG}(a) = H(p) - \Big[P_c \, H\big(p'_{\text{correct}}\big) + (1 - P_c) \, H\big(p'_{\text{wrong}}\big) \Big], \tag{1}$$

  where P_c is the probability of a correct response, and p'_{correct} and p'_{wrong} are the updated mastery probabilities for each possible response. The selected exercise is

$$a^* = \arg \max_{a \in \mathcal{A}_{\text{unasked}}} \text{EIG}(a).$$

  The diagnostic sets $T_{\text{diag}} = 0$ to avoid modelling learning during the test. Ties are broken by the difficulty closest to 0.5, then by lexicographic `activity_id` to ensure deterministic selection.
- **Stopping:** stop when the item cap $N_{\max}$ is reached (proposed 8 items overall and at most 3 per concept), or when every concept in scope has $H(p) \leq 0.50$, that is $p \leq 0.11$ or $p \geq 0.89$.
- **Baselines:** fixed-order and random item selection at the same item budget, used for comparison.

2.4 M3: Time-Budgeted Planner

The planner is a deterministic scheduler with:

1. **Hard filter.** An activity is *eligible* only if all its prerequisite concepts are evidence-mastered and its duration fits in the remaining time of the day. Durations are indivisible.
2. **Ranking.** Eligible activities receive a score

$$\text{score} = w_r \cdot \text{review urgency} + w_u \cdot \text{unlock count} + w_d \cdot \text{difficulty fit},$$

   where the unlock count is the number of downstream concepts that depend on the activity's concept. Ties are broken by `activity_id`.
3. **Packing.** Each day is filled in ranked order without ever exceeding the daily budget $B \in \{5, 10, 15, 30\}$ minutes.
4. **Feasibility.** Cheap bounds are checked first: the critical-path time of unmastered concepts against the time remaining before the deadline, and any mandatory lesson longer than the daily budget. A small exact oracle then verifies small instances.

The oracle gives a three-valued answer, `FEASIBLE`, `INFEASIBLE` or `UNKNOWN`. For example, three activities of $8+8+4$ minutes under a 10-minute budget pass the simple time bounds but cannot fit in two days, so the oracle reports `INFEASIBLE` because the workload needs three days. If the search exceeds its time limit it returns `UNKNOWN` and never a false `INFEASIBLE`. An infeasible result always carries the shortfall and corrective options, such as a longer deadline or a larger budget.

2.5 M4: Mastery Model, Evidence State and Difficulty Policy

2.5.1 Mastery model

A single BKT-style model is used for every concept, so that one consistent state representation drives diagnosis, planning and review. Let p be the current probability that the learner has mastered a concept. For an exercise of difficulty d, the guess and slip probabilities are

$$g(d) = 0.30\,(1 - d), \qquad s(d) = 0.05 + 0.25\,d,$$

so easy items are more likely to be answered correctly without mastery, and hard items are more likely to be missed even with mastery. The predicted probability of a correct answer is

$$P_c(p, d) = p\,(1 - s) + (1 - p)\,g.$$

This quantity is used for next-item selection and for calibration (Brier score). After a response, the posterior is

$$p'_{\text{correct}} = \frac{p(1-s)}{p(1-s) + (1-p)g}, \qquad p'_{\text{wrong}} = \frac{ps}{ps + (1-p)(1-g)}.$$

A learning transition then models the effect of practice:

$$p'' = p' + (1 - p')\,T, \qquad T_{\text{diag}} = 0,\ T_{\text{exercise}} = 0.05,\ T_{\text{lesson}} = 0.20.$$

Worked reference. For $p = 0.5$ and $d = 0.5$ we get $g = 0.15$ and $s = 0.175$. A wrong answer gives a Bayes posterior of $p' \approx 0.1707$; applying the exercise learning transition ($T = 0.05$) gives $p'' \approx 0.2122$. The engine stores the final value p'' in learner state. Tests check both the posterior and the final value, so the Bayes formula and the full update are each verified separately.

Parameter status. The values $p_0 = 0.20$, $\theta = 0.85$, T, $g(d)$ and $s(d)$ are *synthetic heuristics*, not fitted parameters, and all of them are kept in one immutable configuration. Here $\theta = 0.85$ is the *mastery threshold*: a concept is considered mastered only when its mastery probability reaches θ (and enough observed responses exist).

2.5.2 Evidence state

A lesson is instruction, not a measurement, so completing a lesson must not count as assessed mastery. Each learner–concept pair therefore keeps three separate pieces of information:

(i) **Exposure:** lessons completed. They may raise p through T_{lesson} but never mark the concept as assessed.

(ii) **Observed evidence:** the number and outcomes of exercise and diagnostic responses, each with a unique `attempt_id`.

(iii) **Evidence-mastered:** true only when $p \geq \theta$ *and* at least k observed responses exist (proposed $k = 2$).

Prerequisite gating uses *evidence-mastered* only. A replayed `attempt_id` is ignored, so retries cannot inflate mastery, and an event dated earlier than the last practice day cannot rewind it.

2.5.3 Difficulty policy

The next exercise is not chosen by setting difficulty equal to mastery, because under this response model difficulty and mastery are not on the same scale. Instead the policy uses the response model itself: choose the exercise whose predicted success probability $P_c(p, d)$ is closest to a target value of 0.75, so the learner is challenged but not overwhelmed. Ties follow the same chain as in the diagnostic. After each response the mastery estimate is updated as above. Two consecutive wrong answers on a concept with $p < 0.40$ trigger remediation, and two consecutive correct answers with $p \geq \theta$ mark the concept as evidence-mastered.

2.6 M5: Spaced Review and Remediation

New lessons, remediation and review compete for the same few minutes each day, so a clear priority policy is needed.

- **Forgetting model:** We use a simple model to estimate how much a student may forget over time, $r(t) = 2^{-\Delta t/h}$, where Δt is the number of days since the last practice and h is the number of days required for the retention value to reduce by half. Each concept has its own half-life, stored in the learner's state. This serves as the baseline model. Duolingo's Half-Life Regression may be considered if sufficient training data and suitable input features are available, since features used for language learning may not work directly for other concepts.
- **Review urgency:** a concept becomes a review candidate when its predicted recall r falls below a threshold, and its urgency is weighted by its prerequisite importance (the unlock count).
- **Priority order under one budget:** remediation of failed prerequisites first, then high-urgency reviews, then new lessons.
- **Remediation:** when a failed exercise leaves $p < 0.40$, a remediation lesson is inserted, and after the specified remediation cycles the review state and remediation flag are updated.

2.7 M6: Dynamic Replanning

Four events trigger replanning: a `MISSED` day (treated as exactly zero minutes of activity), a `FAILED_ASSESSMENT`, a `DEADLINE_CHANGED` and a `BUDGET_CHANGED`. Every event is a typed object processed through one boundary, `Engine.record(state, log, event)`, which updates learner state and the execution log together. Replanning recomputes only the remaining horizon, keeps every completed attempt without losing or duplicating it, and returns either a valid revised plan or a proof that the new goal is infeasible.

2.8 M7: Decision Traces and Explanations

Every inclusion, omission, review and difficulty change writes a machine-readable record containing the learner state at decision time, the constraint that applied, the score components, and a reason code. For example, an omitted activity carries a code such as "prerequisite not evidence-mastered" or "already mastered above θ", and a selected activity carries its score breakdown. This keeps every decision traceable to state, graph constraints or a policy score, and lets the explanation coverage metric be computed directly.

2.9 M8: Comparative Evaluation

A transparent simulator is used to generate student responses and track their true mastery levels. Its learning behavior differs from the BKT-style model used by the planner, and at least one additional setting is tested to examine performance under model mismatch. Simulator parameters are selected from ranges reported in public educational datasets, and the source of each parameter is documented.

The proposed method is expected to be compared with five fixed baselines: fixed-order and random diagnostic selection with the same item budget, a static plan based on prerequisite order, a greedy planner without review scoring, and full recomputation during replanning. All methods will be evaluated using the same learner profiles, goals, deadlines, time budgets, and random seeds.

Performance is measured across four areas:

1. **Constraint adherence**, including prerequisite violations, budget violations, and the proportion of feasible plans;
2. **Prediction accuracy**, including diagnostic MAE at the same item count, the Brier score for mastery probability P_c, and AUC for predicting correctness on the next attempt;
3. **Learning outcomes**, including goal mastery by the deadline and retention after review;
4. **Efficiency and explainability**, including replanning runtime and explanation coverage.

2.10 Optional Agentic Extension

Learning-based and agentic components are excluded from the core engine. The agentic layer sits above the deterministic core and is disabled by default, so the system works without network access and provides a deterministic baseline for testing. Four bounded extensions are defined. In each, the agent proposes, reorders or chooses, while the engine validates and decides.

- **A0, Authoring Aid:** an offline assistant that drafts curriculum content such as concept relationships, lesson text and remediation hints. Its output is draft content and must pass

schema validation and human review before entering the live curriculum.

- **A1, Plan Proposer:** proposes a day-by-day plan from learner state, goal, budget and deadline. The engine validates every proposal against prerequisite and budget constraints and falls back to the deterministic planner on rejection.
- **A2, Ranking Policy:** re-ranks activities that have already passed the hard-constraint filter, using feedback from past outcomes. It cannot add activities or change eligibility.
- **A3, Orchestrator:** selects the next strategy (continue the plan, remediate, review or re-diagnose) from a closed set of options generated and validated by the engine.

3 System Design

The design follows one rule throughout: the deterministic engine is the authority over validity, mastery, feasibility and state transitions, and everything else works around it.

3.1 Design Principles

Table 1: Architecture quality attributes, also used as review criteria.

#	Attribute	Meaning
Q1	Constraint safety	Prerequisites and daily budgets are enforced as hard constraints by the engine.
Q2	Determinism	The same inputs and configuration always produce the same decisions and traces.
Q3	Traceability	Every decision includes a machine-readable reason.
Q4	Testability	Modules can be tested independently with deterministic results.
Q5	Replaceability	Mastery, review, ranking and planning components can be replaced through defined interfaces.
Q6	Measurable performance	Important operations can be benchmarked on fixed workloads.
Q7	Bounded autonomy	Optional agents may propose or rank options but cannot bypass the engine's constraints.

3.2 System Architecture

AdaptEngine follows a layered architecture. Each layer has one responsibility and depends only on the layers below it.

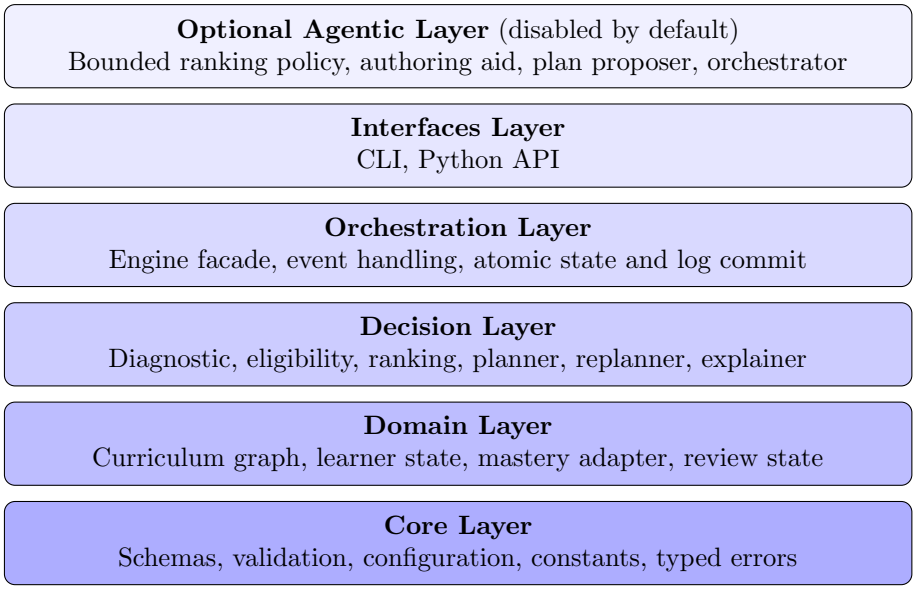


Figure 1: Layered architecture. Each layer uses only the layers below it; the agentic layer sits outside the deterministic core.

Core. Typed schemas, strict validators, configuration, constants and the error hierarchy, with no learning or planning logic. Parameters such as p_0, θ, T and the difficulty bounds live in one immutable configuration whose SHA-256 hash is saved with every run.

Domain. The curriculum DAG, learner state with evidence fields, the mastery adapter, and review state (half-life, last practice day). Display-only fields such as concept descriptions cannot change the graph.

Decision. Diagnostic item selection, eligibility, ranking, planning, replanning, review decisions and explanations, working on validated domain state. Hard constraints are never left to a heuristic or learned ranking.

Orchestration. The Engine facade receives typed events and applies the state transition through the single mutation interface

$$(\text{state}', \log') \leftarrow \texttt{Engine.record}(\text{state}, \log, \text{event}),$$

committing state and log together as one atomic snapshot.

Interfaces. A thin CLI and a Python API only translate external input into typed engine operations and show results or errors, so the same engine serves the CLI, Python programs and the simulator.

Optional agentic layer. Disabled by default so the system works offline and has a deterministic baseline. In each extension the agent proposes, reorders or chooses, while the engine validates and decides:

- **A0 Authoring Aid:** offline drafts of curriculum content; must pass schema validation and human review.
- **A1 Plan Proposer:** proposes plans; the engine validates them and falls back to the deterministic planner on rejection.
- **A2 Ranking Policy:** re-ranks only already-eligible activities.
- **A3 Orchestrator:** picks a strategy from a closed set generated and validated by the engine.

3.3 System Context

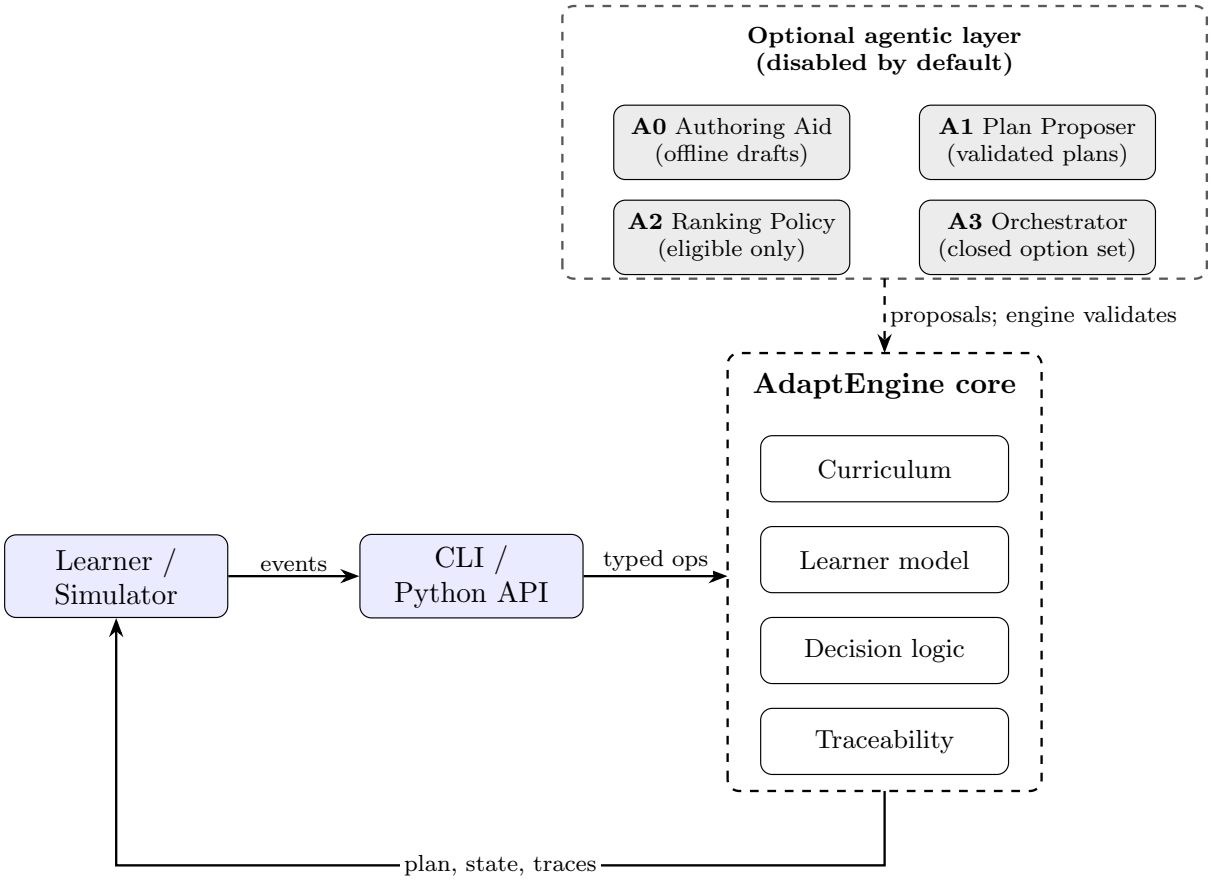


Figure 2: System context. Learners or the simulator interact through a thin CLI or API.

3.4 Major Components and Status

Table 2: Major components and their status at the end of Week 3.

Component	Function	Status
Schemas and validation	Typed dataclasses, strict enums and booleans, finite-probability and range checks, typed errors, configuration hash.	Completed
Curriculum loader	Parses JSON, validates fields, rejects unknown prerequisites and cycles, builds the NetworkX graph.	Completed
Review gate	SHA-256 manifest check, literal `edges_reviewed`, named human sign-off; enforced at every live entry point.	Completed
Learner state and mastery (M4)	Immutable state, evidence fields, attempt IDs, Bayes update, pinned pyBKT adapter with parity test.	Completed; difficulty policy in W4

Component	Function	Status
Diagnostic (M2)	Expected-information-gain selection, caps, stopping rule, descendant skipping, cold start.	Implemented
Atomic persistence	One snapshot for state and log, temp file then replace, failure-injection test.	Completed
Engine facade	`Engine.record(state, log, event)` as single mutation boundary.	Completed
CLI	Thin wrapper over the API; review enforced by default.	Completed
Eligibility and ranking (M3)	Hard filter, score components, deterministic tie-break.	Pending (W4)
Planner and oracle (M3)	Day-by-day packing, feasibility bounds, exact oracle.	Pending (W4)
Review and remediation (M5)	Half-life decay baseline, priority policy under one budget.	Pending (W5)
Replanner (M6)	Handlers for four event types, completed attempts preserved.	Pending (W5)
Explainer (M7)	Decision-trace records.	Pending (W5–6)
Simulator and evaluator (M8)	Transparent simulator, frozen baselines, paired workloads, metrics.	Pending (W6–7)

3.5 Component Interaction and Workflow

1. **Load and validate.** Files are schema-checked, unknown references and cycles are rejected, and the review manifest is verified. A failure stops here with a typed error and no change to state or log.
2. **Diagnose.** The adaptive diagnostic selects a bounded number of items and produces an initial mastery vector with entropy and `UNASSESSED` flags.
3. **Plan.** The planner filters eligible activities, ranks them, packs each day within the budget, and returns a plan or a feasibility verdict, with a decision trace.
4. **Execute and update.** Each outcome is a typed event handled by `Engine.record`, which updates mastery and the log together.
5. **Replan.** A missed day, failed assessment, deadline change or budget change triggers replanning over the remaining horizon.
6. **Evaluate.** The same engine runs against the simulator with fixed seeds and workloads and is compared with frozen baselines.

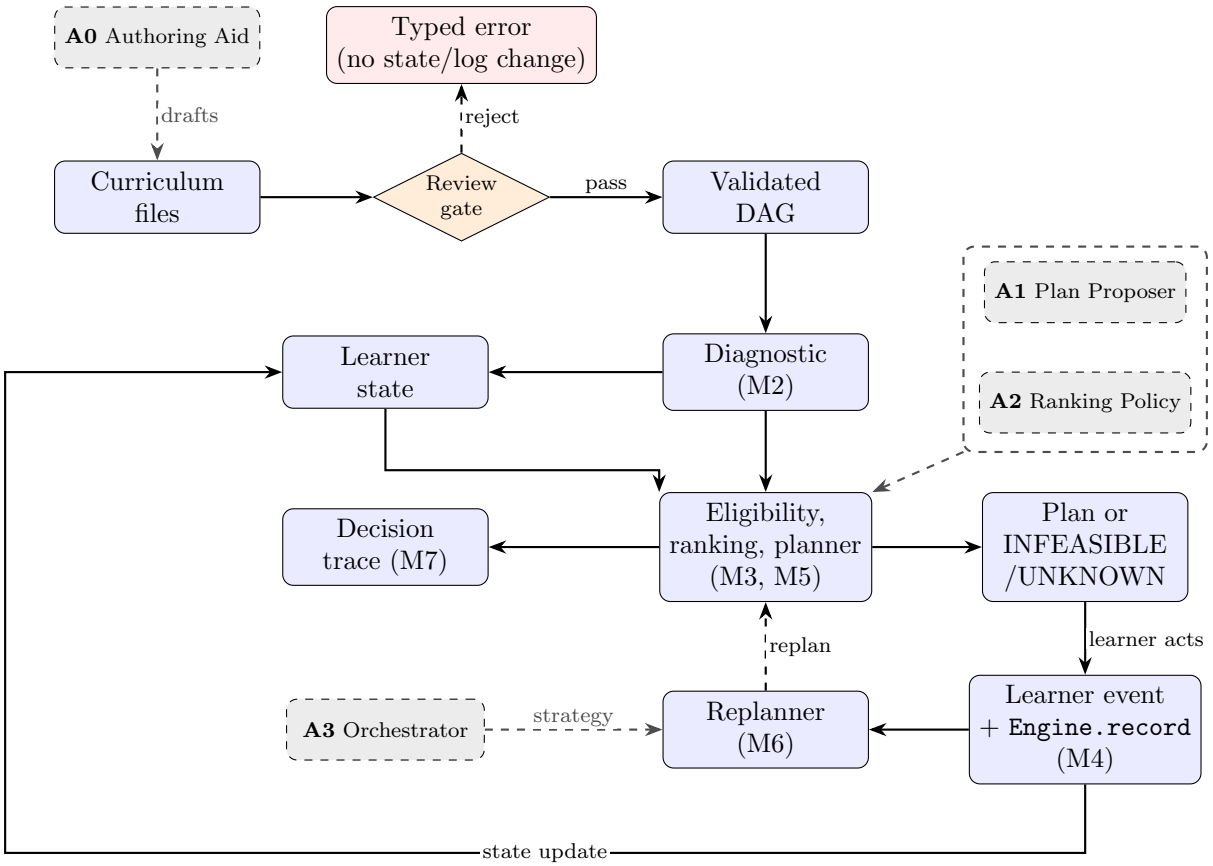


Figure 3: End-to-end workflow.

3.6 Inputs and Outputs

Table 3: Expected inputs and outputs of each operation.

Operation	**Inputs**	**Outputs**
Content load	Curriculum JSON, activities JSON, review manifest, configuration.	Validated DAG and activity table, or a typed error.
Diagnostic	Learner state, goal concepts, item cap $N_{\max}$.	Mastery p, entropy H, assessed flags, item log, stop reasons.
Planner	Learner state, goal concepts, deadline in days, daily budget $B \in \{5, 10, 15, 30\}$.	Day-by-day schedule, or `INFEASIBLE` with shortfall and corrective options, or `UNKNOWN`; decision trace.
Event recording	Typed event with `attempt_id`, activity, strictly boolean correctness, day.	New state and appended log, or a typed error with state and log unchanged.
Replanning	Event and current plan.	Revised valid plan preserving completed attempts, or an infeasibility proof.

Operation	Inputs	Outputs
Evaluation	Simulator profile, seed, workload, baseline name.	Raw run output, configuration hash, code SHA, metrics.

Boundary rules. At every public entry point: content review is required by default; all validation happens before any update, so a rejected input leaves learner and log unchanged; event kinds are strict enumerations and correctness must be a real boolean; unknown prerequisite references are rejected before graph edges are created.

3.7 Data Model

- **Concept:** `id`, `description`, `prerequisites` (existing concept IDs only).
- **Activity:** `id`, `type` (`LESSON` or `EXERCISE`), `target_concept` (exactly one), `duration` (integer minutes, indivisible), `difficulty` in $[0.05, 0.95]$ for exercises.
- **LearnerConcept:** mastery p, lessons done, responses, correct responses, status, half-life, last practice day, seen `attempt_id`s.
- **Event:** strict kind (completed, missed, failed assessment, deadline changed, budget changed), `attempt_id`, payload validated per kind.
- **Plan:** ordered mapping from day to activities, plus feasibility status and decision-trace log.

4 Implementation

This section describes what has been implemented until Week 3.The work was done in two stages: a foundation built in Weeks 0–2, and in Week 3 a repair of that foundation following the mentor review together with the adaptive diagnostic (M2).

4.1 Work Completed Until Week 2: Foundation

- **Problem boundary and decisions.** The scope, literature and data choices were fixed, and the layered architecture was defined.
- **Curriculum.** A four-concept Python curriculum with 20 activities (lessons and exercises, each with an integer duration and, for exercises, a difficulty) was written as JSON and loaded into a NetworkX prerequisite graph.
- **Learner state.** An immutable learner state holds the mastery probability of each concept.
- **Mastery update.** A BKT-style response update was implemented, with guess and slip probabilities depending on item difficulty and a learning transition after practice.
- **Events and engine.** Typed events and the single mutation boundary `Engine.record(state, log, event)` update learner state and execution log together.
- **Interface.** A thin command-line interface (CLI) over the engine.
- **Content review and reproducibility.** A review manifest for curriculum content and a SHA-256 hash of the configuration.

4.2 Work Completed in Week 3

4.2.1 Fixes following the Week 2 mentor review

- **Mandatory review gate.** Review is now required by default at every live entry point (API and each CLI command); loading unreviewed content needs an explicit, clearly named authoring-only mode.
- **Strict validation.** Central validators reject non-boolean responses (the string `"false"`), unknown event kinds, unknown prerequisite references, and non-finite or out-of-range values before any change to state or log.
- **Evidence-state semantics.** Lessons no longer mark a concept as assessed. Learner state separates exposure (lessons), observed evidence (responses with unique `attempt_id`s) and evidence-mastery ($p \geq \theta$ with at least 2 responses), and replayed attempts are ignored.
- **Atomic persistence.** State and log are saved as one snapshot, written to a temporary file and then replaced, and checked by a failure-injection test.
- **Model reuse.** A pinned pyBKT adapter was added behind a common `MasteryModel` interface, with the analytical update kept as an oracle. Difficulty is mapped to three classes (easy, medium, hard) with representative difficulties 0.20, 0.50 and 0.80, where the adapter matches the oracle within 10^{-9}.

4.2.2 Adaptive diagnostic (M2)

The diagnostic is a new `diagnostic/` package that depends on the learner layer only through the `MasteryModel` interface. For the active concept it scores every unasked exercise by expected information gain (EIG) with the learning transition switched off, picks the best item, and updates the belief. The implemented parameters are:

- global cap of 8 items and a cap of 3 items per concept;
- entropy stop when $H(p) \leq 0.50$;
- descendant skipping: if a concept ends with $p < 0.40$, its descendants are not probed;
- cold start: unprobed concepts keep $p_0 = 0.20$, are marked `UNASSESSED` and treated as unmastered, and existing learner state is never overwritten;
- deterministic ties: smaller $|d - 0.5|$, then lexicographic `activity_id`;
- every run ends with one stop reason: `ENTROPY`, `CONCEPT_CAP`, `GLOBAL_CAP`, `NO_ITEMS` or `SKIPPED_ANCESTOR_UNMASTERED`.

One observed property: under the current guess and slip functions, EIG tends to be largest for the hardest items, because a correct answer to a hard item is strong evidence of mastery when guessing is rare. This is a property of the heuristic parameters and is recorded as a limitation.

5 Current Progress

5.1 Milestone Progress

Table 4: Milestone status at the end of Week 3.

Milestone	Status	Note
M1	Done	Boundaries validated and review enforced; TC1.1 and TC1.2 cover loading and rejection.
M2	partial	Caps, stopping, skipping and cold start; TC2.3 depends on the planner and moves to Week 4.
M3	Pending	Planner, bounds and oracle in Week 4.
M4	Partial	Mastery update and adapter done; next-item difficulty policy in Week 4.
M5	Pending	Week 5.
M6	Pending	Week 5.
M7	Pending	Weeks 5–6.
M8	Pending	Weeks 6–7.

5.2 Acceptance Test Register

This record is the register of 23 core test cases. Each ID states its milestone and sequence number (TC4.2 is the second test case of M4).

Table 5: Acceptance test register with current status.

ID	Scenario	Expected result	Status
M1 – Curriculum and learner state			
TC1.1	Inject a cycle into the prerequisite graph.	Cycle rejected at load with a typed error naming the concepts.	Done (W2)
TC1.2	Valid and invalid curriculum, state, event and configuration fixtures; unreviewed content.	Valid load; invalid or unreviewed rejected before any change to state or log.	Done (W3)
M2 – Adaptive diagnostic			
TC2.1	Diagnostic on the four-node fixture with synthetic learners.	Stops at the threshold or cap, never above $N_{\max}$; error no worse than fixed and random selection.	Cap and stop done (W3); comparison in M8
TC2.2	Concepts with no previous observations.	Prior preserved, `UNASSESSED`, treated as unmastered.	Done (W3)
TC2.3	Two learners with different prior mastery, same goal and budget.	Different, individually valid plans.	W4
M3 – Planning and feasibility			
TC3.1	Unmet mandatory prerequisite.	No lesson for the locked concept before its prerequisite is evidence-mastered.	W4

ID	Scenario	Expected result	Status
TC3.2	Fixed daily budget.	No day exceeds the budget.	W4
TC3.3	Same learner and goal, 5 versus 30 min budget.	Materially different pacing or number of days.	W4
TC3.4	Goal cannot fit before the deadline.	`INFEASIBLE` with shortfall and corrective options.	W4
TC3.5	$8 + 8 + 4$ packing fixture, 10 min budget.	Bounds pass but oracle reports `INFEASIBLE`.	W4
TC3.6	Search forced past its time limit.	`UNKNOWN`, never a false `INFEASIBLE`.	W4
M4 – Online mastery update and difficulty			
TC4.1	Incorrect response at $p = 0.5$, $d = 0.5$; adapter versus library.	0.1707 before and 0.2122 after the transition; adapter matches library.	Done (W3)
TC4.2	Select next exercise for a given mastery.	Predicted success nearest the target; ties by rule.	W4
M5 – Review and remediation			
TC5.1	Concept already evidence-mastered above θ.	Omitted from new learning; trace cites the evidence.	W5
TC5.2	Failed exercise leaves $p < 0.40$.	Remediation lesson inserted; state and flag updated.	W5
TC5.3	High review urgency with spare capacity.	Review takes priority over lower-urgency new content.	W5
M6 – Replanning			
TC6.1	Learner misses a scheduled day.	New valid plan; completed work not lost or duplicated.	W5
TC6.2	Failed-assessment event.	State updated; valid revised plan with remediation.	W5
TC6.3	Deadline changed after planning.	Valid revised plan or proof of `INFEASIBLE`.	W5
TC6.4	Daily budget changed.	Valid revised plan satisfying the new budget.	W5
M7 – Decision traces			
TC7.1	Plan with selected and omitted activities.	Every decision traceable to state, constraints and score components.	W5–6
M8 – Evaluation			
TC8.1	Run the harness twice with identical seeds and configuration.	Identical raw results; frozen baselines unchanged.	W6
TC8.2	Frontier replanning versus full recomputation.	Identical decisions; latency recorded for both.	W6–7

6 Assumptions and Limitations

Assumptions. The prerequisite graph is an accurate DAG with hard edges; activities have fixed indivisible durations and a missed day means zero minutes; each exercise maps to exactly one concept; difficulty is continuous in $[0.05, 0.95]$; responses are strictly binary; mastery is a hidden state estimated only from responses.

Table 6: Challenges, risks and mitigations.

Challenge	**Risk**	**Mitigation**
Heuristic parameters	p_0, θ, T, $g(d)$, $s(d)$ are not fitted.	State as synthetic; sensitivity analysis; no fitted-performance claim.
Scripted personas	The diagnostic demo uses hand-written learners; quality against fixed and random selection is not yet measured.	Measure in the M8 harness (W6).
Simulator bias	A simulator built on the planner's rules would inflate results.	Different dynamics family, mismatched variants, held-out profiles.
Review model	Language-learning half-life features may not transfer.	Fixed half-life baseline, clearly labelled.
Binary, indivisible activities	Partial credit, hints and paused lessons are ignored.	Stated assumption and prototype limitation.

7 Future Work

The foundation (M1), mastery model with its adapter, and adaptive diagnostic (M2) are complete. The remaining work includes:

- **Planning and feasibility (M3, M4):** Implement the first static plan, diagnostic-driven eligibility filtering and ranking, day-by-day packing for 5, 10, 15 and 30 minute budgets, the next-item difficulty policy, and feasibility bounds using the exact oracle (`FEASIBLE`, `INFEASIBLE`, `UNKNOWN`).
- **Review and replanning (M5, M6):** Implement the half-life review baseline and urgency score, define the priority order for remediation, review and new lessons under a shared budget, and support replanning after a missed day, failed assessment, deadline change or budget change.
- **Traceability and evaluation (M7, M8):** Generate decision traces for every inclusion, omission, review and difficulty change. Build the transparent simulator, freeze baselines and paired workloads, and compare performance against fixed and random selection.
- **Robustness and agentic extensions:** Evaluate mismatched simulator variants, run sensitivity analysis on heuristic parameters, demonstrate at least one measured improvement over a baseline, and implement agentic extensions.
- **Finalization and documentation:** Deliver the final demonstration, prepare the evaluation report with limitations, and clean up the documentation.